

03

İlk Angular Uygulaması, CLI & DevTools

🕒 Başlangıç · Sıfır → Junior

🔗 Bu modülde ne öğreneceğiz?

- ✓ `ng new` ile bir proje oluşturmayı ve CLI'nin sorduğu soruları
- ✓ Oluşan proje yapısının her dosyasının ne işe yaradığını
- ✓ En çok kullanılan Angular CLI komutlarını
- ✓ `main.ts` + `app.config.ts` ile uygulamanın nasıl başladığını (bootstrapping)
- ✓ Angular DevTools ile uygulamayı incelemeyi
- ✓ v21'in MCP Server'ını ve AI'yı güncel tutmayı; sonunda bir Counter App yapmayı

Ön koşul: Modül 1–2. Geliştirme ortamını (Node 22 LTS, Angular CLI 21.x, VS Code) Modül 1'de kurmuştuk. Bu modülde artık **gerçek bir Angular projesinin** içine giriyoruz: nasıl oluşur, hangi dosyalardan oluşur, nasıl çalışır.



Ders 3.1

ng new ile Proje Oluşturma

Bir Angular projesi her zaman `ng new` ile başlar. Bu tek komut senin için çok iş yapar: bir **workspace** (çalışma alanı) ve içinde çalışmaya hazır bir iskelet uygulama oluşturur, bağımlılıkları (`npm install`) kurar ve bir **git deposu** başlatır.

Komut

```
ng new merhaba-angular
```

Çalıştırınca CLI sana birkaç soru sorar. v21’de tipik sorular şunlardır:

CLI’nin Sorduğu Sorular

Soru	Ne seçmeli?
Stil formatı? (CSS / SCSS / Sass / Less)	SCSS öner — değişken, iç içe yazım gibi ekstralar verir; CSS’in üst kümesidir.
SSR/SSG istiyor musun? (Server-Side Rendering)	Başlangıçta No . SSR’yi Modül 25’te ele alacağız.
AI araç yapılandırması? (claude, copilot, cursor, gemini... / none)	Kullandığın AI aracını seç (yoksa none). Seçersen, AI’a Angular kurallarını öğreten dosyalar üretir.

Zoneless artık varsayılan

Angular v20’de CLI “zoneless ister misin?” diye sorardı. v21’de **zoneless zaten varsayılandır**; yeni projeler `zone.js` olmadan gelir, bu soru çoğunlukla sorulmaz. Yani modern başlangıç hazır.

İşini Hızlandıran Flag’ler

Soruları tek tek cevaplamak yerine komutta belirtebilirsin:

```
ng new merhaba-angular --style=scss --ssr=false --routing
```

Faydalı diğer flag'ler:

```
# --prefix=app      selector ön eki (örn. <app-...>)
# --skip-tests      .spec dosyaları üretme
# --dry-run         hiçbir şey yazmadan ne yapacağını gösterir
# --no-create-application boş workspace (çoklu proje için)
```

🔗 `--dry-run` alışkanlığı

Bir komutun ne üreteceğinden emin değilsen sonuna `--dry-run` ekle. CLI hiçbir dosyaya dokunmadan “şunları oluşturdum” listesini gösterir. Öğrenirken çok güvenli bir alışkanlıktır.

Sonrası

```
cd merhaba-angular
ng serve
# Tarayıcıda http://localhost:4200
```

Birkaç dakika içinde ilk uygulaman ayakta. Bir sonraki derste bu projenin içindeki her dosyayı tek tek tanıyacağız.

🔗 Mini Quiz

1. `ng new` hangi üç işi tek seferde yapar?
2. Stil formatı sorusunda neden `SCSS` önerilir?
3. v21’de “zoneless ister misin?” sorusu neden genelde sorulmaz?
4. `--dry-run` ne işe yarar?

🔗 Cevaplar

1. Workspace + iskelet uygulama oluşturur, bağımlılıkları kurar (`npm install`), git deposu başlatır.
2. SCSS, CSS’in üst kümesidir; değişken/iç içe yazım gibi ekstralar sunar, CSS bilgini de korur.
3. Çünkü v21’de zoneless artık varsayılan; yeni projeler zone.js olmadan gelir.
4. Komutun dosya yazmadan ne yapacağını önizlemeni sağlar.

⊗ Anti-Patterns

- ✗ **Workspace içinde tekrar `ng new` çalıştırmak.** İç içe workspace karışıklık yaratır; yeni proje için üst klasöre çık.
- ✗ **Başlangıçta SSR'yi açmak.** Gereksiz karmaşıklık; temel kavramları otururken No de.
- ✗ **Soruları okumadan geçmek.** Stil formatı ve AI seçimi sonradan etkili; bilinçli seç.

☑ Bu Dersten Çıkarılacaklar

- ✓ `ng new` = workspace + iskelet uygulama + bağımlılıklar + git, tek komutta.
- ✓ v21 soruları: stil (SCSS öner), SSR (başta No), AI araç yapılandırması.
- ✓ Zoneless v21'de varsayılan; ekstra ayar gerekmez.
- ✓ Flag'lerle (`--style` , `--routing` , `--dry-run`) akışı hızlandır.
- ✓ Oluşturduktan sonra `cd` + `ng serve` ile çalıştır.



Ders 3.2

Proje Yapısı Anatomisi

İlk kez bir Angular projesi açınca onlarca dosya görmek korkutucu olabilir. Ama aslında düzen çok mantıklı. Hadi parçalayalım.

Genel Görünüm

```

merhaba-angular/
├── public/                # Statik dosyalar (favicon, görseller)
├── src/                  # ASIL KODUN burada
│   ├── app/             # Uygulama bileşenleri
│   │   ├── app.ts        # Kök bileşen (class App)
│   │   ├── app.html      # Kök bileşenin template'i
│   │   ├── app.css       # Kök bileşenin stili
│   │   ├── app.config.ts # Uygulama yapılandırması (providers)
│   │   ├── app.routes.ts # Rota tanımları
│   │   └── app.spec.ts   # Kök bileşenin testi
│   ├── index.html        # Tek HTML sayfası (SPA kabuğu)
│   ├── main.ts           # Uygulamanın başladığı yer
│   └── styles.scss       # Global stiller
├── angular.json          # Angular CLI yapılandırması
├── package.json          # Bağımlılıklar ve script'ler
├── tsconfig.json         # TypeScript ayarları
└── .gitignore

```

Kök (Workspace) Dosyaları

Dosya	Ne işe yarar?
<code>angular.json</code>	CLI'nin yapılandırması: build/serve/test ayarları, stil dosyaları, çıktı yolu.
<code>package.json</code>	Bağımlılıklar (Angular paketleri vb.) ve <code>npm</code> script'leri.
<code>tsconfig.json</code>	TypeScript derleyici ayarları (<code>tsconfig.app.json</code> ve <code>tsconfig.spec.json</code> bundan miras alır).
<code>public/</code>	Olduğu gibi sunulan statik dosyalar (favicon, resimler). Eski <code>src/assets</code> 'in yerini aldı.
<code>.gitignore</code>	<code>node_modules</code> gibi git'e gönderilmeyecek klasörler.

src/ — Asıl Kod

- `main.ts` — **giriş noktası**. Uygulamayı başlatır (bootstrapping). Detayı Ders 3.4'te.
- `index.html` — SPA'nın tek HTML kabuğu. İçinde genelde sadece `<app-root></app-root>` vardır; gerisini Angular doldurur.
- `styles.scss` — tüm uygulamaya uygulanan **global** stiller.
- `app/` — bileşenlerin yaşadığı klasör.

app/ — Kök Bileşen

Bir Angular bileşeni genelde üç-dört dosyadan oluşur: mantık (`.ts`), template (`.html`), stil (`.css` / `.scss`) ve test (`.spec.ts`). Kök bileşen `app.ts` şuna benzer:

TYPESCRIPT

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet], // standalone: ihtiyaçları doğrudan import eder
  templateUrl: './app.html',
  styleUrls: ['./app.css'],
})
export class App {
  protected readonly title = signal('merhaba-angular');
}
```

ⓘ Dikkat: NgModule yok!

Eski Angular projelerinde bir `app.module.ts` ve `NgModule` olurdu. Modern (standalone) Angular'da **yok**. Her bileşen ihtiyaçlarını kendi `imports` dizisinde belirtir. İnternette `NgModule` 'lü örnek görürsen, o eski yöntemdir.

Bir İsim Detayı: “2025” Stili

v21 varsayılan olarak **sade dosya adları** kullanır: `app.ts`, `app.html` ve sınıf adı `App`. Eski (“2016”) stilde bunlar `app.component.ts` ve `AppComponent` şeklindeydi. İkisi de aynı işi yapar; v21 kısa olanı tercih eder. Eski eğitimlerde `.component` ekini görürsen şaşırma.

❓ Mini Quiz

1. Uygulamanın giriş noktası (başladığı dosya) hangisidir?
2. `index.html` içinde genelde ne bulunur?
3. Modern Angular'da `app.module.ts` / `NgModule` var mı?
4. Statik görseller hangi klasöre konur?

🔑 Cevaplar

1. `main.ts`.
2. Genelde sadece `<app-root></app-root>`; içeriği Angular doldurur.
3. Hayır. Standalone yapıda yok; her bileşen kendi `imports` 'unu belirtir.
4. `public/` klasörü (eski `src/assets` 'in yerini aldı).

⊗ Anti-Patterns

- ✗ `node_modules` 'ı git'e göndermek. Çok büyüktür ve `.gitignore` zaten dışlar; asla commit'leme.
- ✗ Eski `NgModule` örneklerini bu yapıya yapıştırmak. Standalone projede uyumsuzdur.
- ✗ `angular.json` 'ı rastgele kurcalamak. Güçlüdür ama hassastır; ne yaptığını bilmeden değiştirme.

☑ Bu Dersten Çıkarılacaklar

- ✓ Kod `src/` içinde; `main.ts` giriş, `index.html` SPA kabuğu, `styles.scss` global stil.
- ✓ Bileşen = `.ts` (mantık) + `.html` (template) + `.css` (stil) + `.spec.ts` (test).
- ✓ Kök bileşen `app.ts` / sınıf `App` ; standalone'dır (NgModule yok).
- ✓ Kök dosyalar: `angular.json` , `package.json` , `tsconfig.json` .
- ✓ v21 “2025” stili sade adlar kullanır (`app.ts` , sınıf `App`).



Ders 3.3

Angular CLI Komutları

Angular CLI (`ng`) günlük hayatının en sık kullandığın aracı olacak. Proje oluşturmaktan kod üretmeye, çalıştırmaktan test etmeye kadar her şeyi yapar. En önemlilerini tanıyalım.

Temel Komutlar

Komut	Ne yapar?
<code>ng serve</code>	Geliştirme sunucusunu başlatır (<code>localhost:4200</code>), hot reload'lu.
<code>ng build</code>	Üretim için optimize edilmiş çıktı üretir (<code>dist/</code> klasörüne).
<code>ng test</code>	Testleri çalıştırır (v21'de Vitest ile).
<code>ng generate</code>	Yeni bileşen/servis/pipe vb. üretir (kısa: <code>ng g</code>).
<code>ng add</code>	Bir kütüphaneyi kurup yapılandırır (örn. <code>ng add @angular/material</code>).
<code>ng update</code>	Angular'ı ve bağımlılıkları güvenli şekilde günceller.
<code>ng lint</code>	Kod kalitesi/stil denetimi yapar (ESLint).
<code>ng mcp</code>	MCP sunucusunu başlatır (Ders 3.6).

`ng generate` — En Çok Kullanacağın

Elle dosya oluşturmak yerine CLI üretsinsin; hem hızlı hem doğru yapıda olur:

```
ng generate component features/gorev-listesi
# kısaca:
ng g c features/gorev-listesi

ng g s core/gorev          # service
ng g d shared/vurgu        # directive
ng g p shared/tarih-bicim  # pipe
ng g g core/auth           # guard
ng g interface models/gorev # interface
```

Örneğin `ng g c features/gorev-listesi` şu dosyaları üretir: `gorev-listesi.ts`, `gorev-listesi.html`, `gorev-listesi.css`, `gorev-listesi.spec.ts` — hepsi doğru bağlanmış halde.

🔍 Faydalı flag'ler

`--dry-run` (önizleme), `--skip-tests` (`.spec` üretme), `--flat` (alt klasör açmadan). Örn: `ng g c sayac --dry-run` ile ne üreteceğini önce gör.

Yardım Almak

BASH

```
ng help           # tüm komutlar
ng generate --help # generate seçenekleri
ng serve --help   # serve seçenekleri
```

🔍 Mini Quiz

1. Geliştirme sunucusunu başlatan komut nedir?
2. `ng g c sayac` kaç dosya üretir ve hangileri?
3. v21'de `ng test` hangi test çalıştırıcısını kullanır?
4. Bir komutun ne üreteceğini yazmadan görmek için hangi flag?

🔍 Cevaplar

1. `ng serve`.
2. Dört: `sayac.ts`, `sayac.html`, `sayac.css`, `sayac.spec.ts`.
3. Vitest.
4. `--dry-run`.

⊗ Anti-Patterns

- ⊗ Dosyaları hep elle oluşturmak. `ng generate` doğru isim/yapı/bağlantıyı otomatik kurar; hata payını azaltır.
- ⊗ `ng update` yerine `package.json`'ı elle düzenlemek. Migration adımlarını atlar, projeyi bozabilirsiniz.
- ⊗ `ng build` çıktısını geliştirmede beklemek. Geliştirme için `ng serve` kullan; build üretim içindir.

✔ Bu Dersten Çıkarılacaklar

- ✔ Çekirdek: `ng serve` , `ng build` , `ng test` , `ng generate` , `ng add` , `ng update` .
- ✔ `ng g c <yol/ad>` bileşeni dört dosyayla, doğru bağlanmış üretir.
- ✔ v21'de testler Vitest ile çalışır.
- ✔ `--dry-run` ile güvenli önizle; `ng <komut> --help` ile seçenekleri öğren.
- ✔ Güncellemeyi her zaman `ng update` ile yap.



app.config.ts & Bootstrapping

Bootstrapping, uygulamanın “çalışmaya başlama” anıdır: Angular kök bileşeni alır, tarayıcıdaki `<app-root>` etiketine yerleştirir ve uygulamayı ayağa kaldırır. Bunu yapan yer `main.ts`’tir.

main.ts — Giriş Noktası

TYPESCRIPT

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from '../app/app.config';
import { App } from '../app/app';

bootstrapApplication(App, appConfig)
  .catch((err) => console.error(err));
```

`bootstrapApplication` iki şey alır: **kök bileşen** (`App`) ve **uygulama yapılandırması** (`appConfig`). Eski Angular’da burada `NgModule` ve `platformBrowserDynamic().bootstrapModule(...)` olurdu; modern yol çok daha sade.

app.config.ts — Uygulama Yapılandırması

Uygulama genelinde geçerli ayarları ve **provider’ları** burada toplarsın:

TYPESCRIPT

```
import {
  ApplicationConfig,
  provideBrowserGlobalErrorListeners,
} from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from '../app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(), // global hataları yakalar
    provideRouter(routes),               // yönlendirme (routing)
  ],
};
```

① Zoneless: ekstra provider yok

Eski eğitimlerde burada `provideZonelessChangeDetection()` görebilirsin. v21’de zoneless **varsayılan** olduğu için yeni projelerde bunu eklemene gerek yok. (Tersine, eski bir `zone.js` kütüphanesine ihtiyacın olursa `provideZoneChangeDetection()` eklenir.)

“Provider” Nedir?

Provider, uygulamaya bir **yetenek** veya **servis** sağlamaktır. Modern Angular bunları `provideX()` fonksiyonlarıyla verir. Sık göreceğiniz:

Provider	Ne sağlar?
<code>provideRouter(routes)</code>	Yönlendirme/navigasyon (Modül 9)
<code>provideHttpClient()</code>	Sunucuyla HTTP iletişimi (Modül 11)
<code>provideAnimations()</code>	Animasyon desteği (Modül 15)
<code>provideBrowserGlobalErrorListeners()</code>	Tarayıcıdaki global hataları yakalama

Bu `provide...` deseni sayesinde uygulamana sadece **ihtiyacın olan** parçaları eklersin; gereksiz kod paketine girmez (daha küçük uygulama).

app.routes.ts — Rotalar

Şimdilik kısaca: hangi URL’in hangi bileşeni göstereceğini burada tanımlarsın (detayı Modül 9):

TYPESCRIPT

```
import { Routes } from '@angular/router';

export const routes: Routes = [
  // { path: '', component: AnaSayfa },
  // { path: 'gorevler', component: GorevListesi },
];
```

❓ Mini Quiz

1. Bootstrapping tek cümleyle nedir?
2. `bootstrapApplication` hangi iki şeyi alır?
3. Provider nedir?
4. v21’de `app.config.ts`’e zoneless için ekstra provider gerekir mi?

🔗 Cevaplar

1. Uygulamanın çalışmaya başlaması; kök bileşenin sayfaya yerleştirilip ayağa kaldırılması.
2. Kök bileşen (`App`) ve uygulama yapılandırması (`appConfig`).
3. Uygulamaya bir yetenek/servis sağlama; modern Angular’da `provideX()` fonksiyonlarıyla verilir.
4. Hayır; zoneless varsayılan. (Aksini istersen `provideZoneChangeDetection()` eklenir.)

⊗ Anti-Patterns

- ✗ Eski `platformBrowserDynamic` / `NgModule` bootstrap kodunu kullanmak. Modern yol `bootstrapApplication(App, appConfig)`.
- ✗ Her şeyi global provider yapmak. Sadece gerçekten uygulama geneli olanları `app.config.ts` 'e koy; gerisi bileşen/route seviyesinde.
- ✗ Provider fonksiyonlarını çağırmaıı unutmak. `provideRouter(routes)` (parantezli) yazılır; `provideRouter` (çağrısız) işe yaramaz.

☑ Bu Dersten Çıkarılacaklar

- ✓ Bootstrapping `main.ts` 'te `bootstrapApplication(App, appConfig)` ile olur.
- ✓ `app.config.ts` uygulama geneli provider'ları toplar (`ApplicationConfig`).
- ✓ Provider = uygulamaya yetenek/servis sağlama; modern desen `provideX()` .
- ✓ Zoneless v21'de varsayılan; ekstra provider gerekmez.
- ✓ `app.routes.ts` URL'leri bileşenlere bağlar (detay Modül 9).



Angular DevTools, çalışan uygulamanın “içini görmeni” sağlayan bir tarayıcı eklentisidir. Console’a `console.log` serpiştirmek yerine, uygulamanın yapısını ve verilerini doğrudan inceleyebilirsiniz.

Kurulum

Chrome veya Edge’in eklenti mağazasında “**Angular DevTools**” ara ve ekle. Sonra uygulamayı `ng serve` ile çalıştırıp tarayıcıda aç, `F12` ile geliştirici araçlarını aç — sekmeler arasında bir “**Angular**” sekmesi belirir.

❗ Sadece geliştirme modunda

DevTools yalnızca **geliştirme** derlemesinde (`ng serve`) çalışır. Üretim build’inde (optimizasyon nedeniyle) devre dışıdır — bu normaldir.

Components Sekmesi

Uygulamanın **bileşen ağacını** gösterir. Bir bileşene tıklayınca sağ panelde:

- Bileşenin **property’leri ve signal değerleri** görünür.
- Değerleri **canlı değiştirip** arayüzün nasıl tepki verdiğini görebilirsin.
- Seçili bileşenin kaynağına (`.ts`) atlayabilirsin.

Profiler Sekmesi

Change detection performansını ölçer: kaydı başlat, uygulamada gez, durdur. Hangi bileşenin ne zaman güncellendiğini görürsün. Yavaşlık avında paha biçilmezdir (detay Modül 22). Zoneless dünyada “gereksiz güncellenen” yerleri yakalamak için çok faydalı.

Injector Tree

Dependency Injection (DI) ağacını gösterir: hangi servis nerede sağlanmış, bir bileşen hangi servisleri enjekte etmiş. DI’yı Modül 7’de öğreneceğiz; o zaman bu sekme çok anlamlı olacak.

Günlük Kullanım

1. Bir şey beklediğin gibi görünmüyor mu? Components’ta o bileşenin signal/property değerlerine bak.
2. Uygulama yavaş mı? Profiler ile change detection’ı ölç.
3. “Bu servis nereden geliyor?” Injector Tree’ye bak.

🔍 Mini Quiz

1. Angular DevTools hangi derlemede çalışır: geliştirme mi, üretim mi?
2. Components sekmesinde bir bileşene tıklayınca ne görürsün?
3. Profiler ne ölçer?
4. Console'a `console.log` yığmak yerine neyi kullanmalısın?

🔑 Cevaplar

1. Geliştirme (`ng serve`). Üretim build'inde devre dışıdır.
2. Bileşenin property/signal değerlerini; onları canlı değiştirebilir, kaynağına atlayabilirsiniz.
3. Change detection performansını (hangi bileşen ne zaman güncellendi).
4. Angular DevTools'u (Components/Profiler/Injector Tree).

❌ Anti-Patterns

- ❌ Her şeyi `console.log` ile ayıklamak. DevTools daha hızlı ve net; signal değerlerini canlı gösterir.
- ❌ Üretim build'inde DevTools beklemek. Orada kapalıdır; geliştirme modunda kullan.
- ❌ Profiler'ı hiç açmamak. Performans sorunlarını tahminle değil ölçerek çöz.

✅ Bu Dersten Çıkarılacaklar

- ✓ Angular DevTools, çalışan uygulamayı incelemek için tarayıcı eklentisidir (geliştirme modunda).
- ✓ Components: bileşen ağacı + signal/property değerleri (canlı düzenlenebilir).
- ✓ Profiler: change detection performansı; Injector Tree: DI yapısı.
- ✓ Hata ayıklamada `console.log` yerine ilk durağın DevTools olsun.



MCP Server (v21 Yeniliği)

Modül 1’de tanışmıştık; şimdi pratiğe dökelim. **MCP (Model Context Protocol) Server**, AI asistanların (Claude, Copilot, Cursor, Gemini...) **güncel Angular bilgisine ve senin projenin yapısına** erişmesini sağlayan resmî bir köprüdür.

Hangi Problemi Çözer?

AI modelleri belirli bir tarihe kadarki bilgiyle eğitilir (“knowledge cutoff”). Bu yüzden sana eski Angular (NgModule’lü, eski form) kodu önerebilir. MCP Server, AI’nın **resmî ve güncel** kaynaklardan beslenmesini sağlar; böylece v21 standartlarına (standalone, signals, yeni control flow) uygun, hatta Signal Forms gibi **yepyeni** özelliklere uygun kod üretir.

❗ Bu eğitimle birebir ilgili

Bu ders kitabını okurken AI’dan kod isteyeceksen, MCP’yi kurman “v21 istiyorum ama bana eski kod geliyor” sorununu büyük ölçüde çözer.

Nasıl Kurulur?

İki yol var. Hızlı deneme için terminalden:

```
BASH
ng mcp
```

Kalıcı kullanım için, IDE’nin (Cursor, VS Code, Claude Code...) MCP yapılandırmasına bir `mcp.json` eklenir:

```
JSON
{
  "servers": {
    "angular-cli": {
      "command": "npx",
      "args": ["-y", "@angular/cli", "mcp", "--read-only"]
    }
  }
}
```

💡 `--read-only` ile güvenli başla

`--read-only` bayrağı, sunucunun yalnızca **okuma** yapan araçlarını açar (dosyalarına yazmaz). Başlangıçta en güvenli seçimdir.

v21'deki 7 Araç (6 Stabil + 1 Deneysel)

Araç	Ne yapar?
<code>list_projects</code>	Workspace'teki tüm uygulama ve kütüphaneleri listeler.
<code>get_best_practices</code>	Angular en iyi pratikler kılavuzunu getirir (AI standartlara uysun).
<code>search_documentation</code>	Resmî Angular dokümanını arar (API, kavram, örnek).
<code>find_examples</code>	Güncel, resmî, en iyi pratik örnek kodları bulur.
<code>ai_tutor</code>	IDE içinde adım adım AI öğretmen modunu açar.
<code>onpush-zoneless-migration</code>	OnPush/zoneless'e geçiş için adım adım öneri verir.
<code>modernize</code> (deneysel)	control-flow, signal-input, standalone gibi modern migration'ları çalıştırır (diske yazar).

İlk altısı okuma odaklı (güvenli); `modernize` deneyseldir ve dosyalarını değiştirebildiği için dikkatli kullanılır.

Önerilen Akış

- Önce `list_projects` ile AI projeyi tanısn.
- `get_best_practices` ile standartlara hizalansın.
- İhtiyaca göre `search_documentation` / `find_examples` kullansın.

❓ Mini Quiz

- MCP Server hangi temel problemi çözer?
- MCP'yi yalnızca okuma yapan güvenli modda başlatan bayrak nedir?
- v21 MCP kaç araçla gelir ve kaçı stabil?
- AI'ı Angular standartlarına hizalamak için hangi araç kullanılır?

🔑 Cevaplar

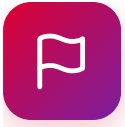
- AI'nın knowledge-cutoff sorununu: güncel/resmî bilgiyle v21'e uygun kod üretmesini sağlar.
- `--read-only`.
- Yedi araç; altısı stabil, biri (`modernize`) deneysel.
- `get_best_practices`.

⊗ Anti-Patterns

- ✗ **AI'a körü körüne güvenmek.** MCP yardımcı olur ama kuralları her zaman tam uygulamaz; ürettiği kodu yine de incele.
- ✗ **Deneyisel `modernize` 'ı temiz olmayan git dalında çalıştırmak.** Diske yazar; önce işini commit'le.
- ✗ **MCP'yi kurup hiç kullanmamak.** En azından `get_best_practices` + `find_examples` ile başla.

☑ Bu Dersten Çıkarılacaklar

- ✓ MCP Server, AI'ı güncel/resmî Angular bilgisine ve projenin yapısına bağlar.
- ✓ Kurulum: `ng mcp` veya IDE'de `mcp.json` (`npx -y @angular/cli mcp --read-only`).
- ✓ v21'de 7 araç: 6 stabil (`list_projects`, `get_best_practices`, `search_documentation`, `find_examples`, `ai_tutor`, `onpush-zoneless-migration`) + 1 deneyisel (`modernize`).
- ✓ `--read-only` ile güvenli başla; `modernize` diske yazar, dikkatli kullan.
- ✓ AI'ın çıktısını her zaman gözden geçir; MCP garanti değil, yardımcıdır.



Modül 1’de üç framework’te gördüğümüz sayacı, şimdi **gerçek bir Angular projesinde** hayata geçiriyoruz. Amaç: CLI ile bileşen üretmek, bir signal kullanmak ve sonucu tarayıcıda çalışır görmek. (Component ve signal’in detayını Modül 4–5’te derinlemesine işleyeceğiz; burada “ilk çalıştırma” deneyimi öncelikli.)

Adım 1 — Proje ve Bileşen

BASH

```
ng new sayac-app --style=scss --ssr=false
cd sayac-app
ng g c sayac          # sayac bileşenini üretir
```

Adım 2 — Bileşen Mantığı (sayac.ts)

TYPESCRIPT

```
import { Component, signal, computed } from '@angular/core';

@Component({
  selector: 'app-sayac',
  templateUrl: './sayac.html',
  styleUrls: ['./sayac.css'],
})
export class Sayac {
  deger = signal(0); // durum
  ikiKat = computed(() => this.deger() * 2); // türetilmiş değer

  artir() { this.deger.update((n) => n + 1); }
  azalt() { this.deger.update((n) => n - 1); }
  sifirla() { this.deger.set(0); }
}
```

`signal(0)` sayacın durumunu tutar; `computed(...)` ona bağlı türetilmiş bir değerdir (değer değişince otomatik güncellenir). `update` mevcut değere göre, `set` doğrudan yeni değer atar.

Adım 3 — Template (sayac.html)

```
<div class="sayac">
  <h2>Sayaç: {{ deger() }}</h2>
  <p>İki katı: {{ ikiKat() }}</p>

  <div class="butonlar">
    <button (click)="azalt()">-</button>
    <button (click)="sifirla()">Sıfırla</button>
    <button (click)="artir()">+</button>
  </div>
</div>
```

Parantezlere dikkat: `{{ deger() }}` (signal fonksiyon gibi çağrılır) ve `(click)="artir()"` (olay dinleme).

Adım 4 — Kök Bileşene Ekle (app.ts & app.html)

```
// app.ts - Sayac'ı imports'a ekle
import { Component } from '@angular/core';
import { Sayac } from './sayac/sayac';

@Component({
  selector: 'app-root',
  imports: [Sayac],
  templateUrl: './app.html',
  styleUrls: ['./app.css'],
})
export class App {}
```

```
<!-- app.html -->
<h1>Counter App</h1>
<app-sayac />
```

Adım 5 — Biraz Stil (sayac.css)

```
.sayac { text-align: center; font-family: sans-serif; }
.butonlar { display: flex; gap: .5rem; justify-content: center; }
button {
  padding: .5rem 1rem; font-size: 1.1rem; cursor: pointer;
  border: 1px solid #ccc; border-radius: 8px; background: #fff;
}
button:hover { background: #f3f3f3; }
```

Adım 6 — Çalıştır

BASH

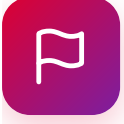
```
ng serve
# http://localhost:4200 – butonlara bas, sayaç ve "iki katı" anında değişsin
```

🔗 Dene & gözlemler

Tarayıcıda butonlara bas; `deger` değişince hem ekran hem `ikiKat` **otomatik** güncelleniyor — işte reactivity (Modül 1). Sonra Angular DevTools'u aç, Components'ta `Sayac`'ı seç ve `deger` signal'ının canlı değiştiğini izle.

✅ Bu Dersten Çıkarılacaklar

- ✓ `ng g c` ile bileşen ürettin, `signal` + `computed` ile durum tuttun.
- ✓ `{{ deger() }}` (signal okuma) ve `(click)` (olay) sözdizimini ilk kez kullandın.
- ✓ Bileşeni kök bileşenin `imports`'una ekleyip `<app-sayac />` ile yerleştirdin.
- ✓ Reactivity'yi canlı gördün: değer değişince ekran ve türetilmiş değer otomatik güncellendi.



Artık bir Angular projesini oluşturabiliyor, içindeki dosyaları tanıyor, CLI ile çalışabiliyor ve uygulamanın nasıl başladığını biliyorsun. İlk gerçek uygulamayı da yaptın.

Öğrendiklerin

- **ng new** : workspace + iskelet uygulama + bağımlılıklar + git; stil/SSR/AI soruları; zoneless varsayılan.
- **Proje yapısı**: `src/`, `main.ts`, `index.html`, `app/` (standalone, `app.ts` / sınıf `App`, NgModule yok).
- **CLI**: `serve` / `build` / `test` / `generate` / `add` / `update`; `ng g c` dört dosyayı üretir; `--dry-run`.
- **Bootstrapping**: `bootstrapApplication(App, appConfig)`; `app.config.ts` provider'ları (`provideRouter` vb.) toplar.
- **Angular DevTools**: Components, Profiler, Injector Tree.
- **MCP Server**: AI'ı güncel/projeye uygun tutan 7 araç; `--read-only` ile güvenli başla.

❓ Sırada ne var?

Modül 4'te işin kalbine giriyoruz: **Component'lar & Modern Template Syntax**. Component anatomisi, standalone yapı, dört çeşit data binding, yeni control flow (`@if`, `@for`), `@let` ve `input()` / `output()` / `model()` 'i öğrenip bir **ToDo App** yapacağız.

💡 Kendini test et

Şunları cevaplayabiliyor musun? “ `ng new` ne üretir? Uygulama nereden başlar? Provider nedir? `ng g c` kaç dosya üretir? DevTools ne işe yarar?” Cevaplayabiliyorsan Modül 4'e hazırsın.

Takıldığın her yeri sor — hazır olduğunda Modül 4'e geçeriz. Kolay gelsin!